

Guía de Estudio Manual: Acceso a Datos

Desarrollo Teórico y Conceptual Completo del Temario

Ciclo Formativo de Grado Superior DAM • Convocatoria: Junio 2026

Propósito Didáctico: Este documento transforma el banco de preguntas sueltas en una guía de desarrollo secuencial y explicativa. Sirve para comprender los fundamentos de la persistencia avanzada, bases de datos nativas XML y el estándar de componentes reutilizables (JavaBeans), articulando la teoría necesaria para el examen.

BLOQUE 1: Modelos de Persistencia Avanzada (BDOO, BDOR y JPA)

1.1 EL DESFASE OBJETO-RELACIONAL Y MODELOS ALTERNATIVOS

En la ingeniería de software moderna, la mayor parte del desarrollo se realiza bajo el paradigma de la **Programación Orientada a Objetos (POO)**. Sin embargo, los sistemas de almacenamiento más extendidos han sido tradicionalmente las bases de datos relacionales. Esta disparidad da lugar al denominado **desfase (o desajuste) objeto-relacional**, que define la dificultad intrínseca, complejidad y pérdida de eficiencia al traducir estructuras de objetos complejas en memoria (con herencia, polimorfismo y encapsulación) a tablas rígidas bidimensionales compuestas por filas y columnas vinculadas mediante claves ajenas.

Para mitigar este desfase se desarrollaron dos enfoques principales de bases de datos:

- **Bases de Datos Orientadas a Objetos (BDOO):** Tienen como principal ventaja que eliminan por completo el desfase relacional, ya que permiten la **persistencia de objetos de forma nativa sin transformaciones** intermedias. Ofrecen una correspondencia directa con el modelo de objetos de la aplicación. Según el estándar **ODMG**, la estructura de una BDOO se define mediante el lenguaje **ODL** (Lenguaje de Definición de Objetos), y sus consultas se ejecutan con **OQL** (Object Query Language). En este entorno, la reutilización de código se apoya en características como la **sobrecarga de operadores**. Un ejemplo ideal integrado con Java es **ObjectDB**, que destaca por omitir los mapeos complejos tradicionales.
- **Bases de Datos Objeto-Relacionales (BDOR):** Es una arquitectura híbrida que **integra conceptos de objetos y tablas** dentro de un motor relacional evolutivo. Permite la declaración de **tipos de datos personalizados** por el usuario (User-Defined Types, UDT) para representar con mayor precisión la lógica del negocio, soporta el uso de **Array** para almacenar colecciones ordenadas dentro de una única columna y permite la relación de herencia entre tablas, dotando al sistema de una alta flexibilidad para el manejo de estructuras de datos complejas.

1.2 ESTÁNDARES SQL:1999 Y EL MOTOR POSTGRESQL

El estándar internacional **SQL:1999 (SQL3)** introdujo estas características híbridas de forma universal, habilitando la creación de **métodos** específicos dentro del motor de base de datos para operar directamente con objetos complejos. Dentro de esta especificación, se estandarizaron tipos especializados para grandes volúmenes de datos: **CLOB** (Character Large Object), idóneo para textos extremadamente extensos, y **BLOB** (Binary Large Object), la unidad de almacenamiento recomendada para datos binarios puros como **imágenes y vídeos**.

PostgreSQL es el sistema gestor de bases de datos de código abierto que mejor representa este modelo objeto-relacional. Es un entorno multiplataforma con soporte total para Unicode y conjuntos de caracteres internacionales. Su arquitectura destaca por dos puntos clave:

1. Soporte nativo de herencia entre tablas, lo que faculta a los desarrolladores para reutilizar estructuras base de datos ya definidas.
2. Implementación del mecanismo **MVCC (Multi-Version Concurrency Control)**, una característica avanzada que permite a PostgreSQL manejar **alta concurrencia de datos sin necesidad de bloqueos** explícitos de tablas, permitiendo que las lecturas y escrituras coexistan simultáneamente sin penalizar el rendimiento. Su administración se realiza de forma visual mediante la utilidad `pgAdmin`, y soporta capacidades críticas empresariales como la **replicación de datos**.

1.3 FRAMEWORKS ORM Y ARQUITECTURA DE JPA

Cuando el almacenamiento sigue siendo relacional estricto, la solución habitual en el desarrollo de software es delegar en un framework de **Mapeo Objeto-Relacional (ORM)** como **Hibernate**, cuyo objetivo es simplificar drásticamente la persistencia mediante la automatización del traslado de datos entre el código y la base de datos.

En el ecosistema Java, el estándar oficial para coordinar este flujo es la especificación **JPA (Java Persistence API)**. El elemento modular encargado de gobernar el ciclo de vida de los objetos persistentes y sus interacciones es la interfaz **EntityManager**.

```
[Objeto Transient] -- em.persist() --> [Estado MANAGED] -- em.remove() --> [Estado REMOVED]
                        ^                                     |
                        |----- em.merge() ----- [Estado DETACHED] <--- Transacción
                                                finalizada
```

El **EntityManager** orquesta las operaciones de datos fundamentales en la aplicación a través de métodos predefinidos:

- **persist()**: Almacena de forma efectiva una entidad transitoria en el contexto de persistencia, traduciendo en una inserción en la base de datos.
- **merge()**: Actualiza el estado de una entidad en la base de datos sincronizando los cambios de un objeto desasociado.
- **find()**: Recupera del almacenamiento una entidad específica basándose de forma directa en su clave primaria.
- **remove()**: Marca un objeto gestionado para su eliminación física del almacenamiento relacional.

Para garantizar la integridad total de los datos procesados, estas operaciones se agrupan en transacciones que aseguran el cumplimiento de las **propiedades ACID** (Atomicidad, Consistencia, Aislamiento y Durabilidad). El método **commit()** de la transacción es el responsable directo de persistir de forma definitiva las modificaciones en el disco garantizando estas propiedades. Asimismo, para realizar búsquedas dinámicas avanzadas independientes del motor subyacente, JPA incorpora el lenguaje **JPQL** (Java Persistence Query Language), cuyas consultas se ejecutan sobre las entidades orientadas a objetos y no sobre las tablas físicas relacionales.

BLOQUE 2: Almacenamiento y Consulta de Documentos XML (BDB XML y XQJ)

2.1 ARQUITECTURA DE BASES DE DATOS XML: NATIVA VS. ENABLED

Con la proliferación de documentos semiestructurados en formato XML, la persistencia ha requerido adaptar su infraestructura en dos grandes variantes: los sistemas habilitados para XML (**XML-Enabled**) y las bases de datos **XML Nativas**.

La diferencia crítica radica en que las bases de datos *XML-Enabled* no son más que motores relacionales tradicionales adaptados que **necesitan un mapeo previo o intermedio** para fragmentar y encajar el documento XML en un esquema de tablas. Por el contrario, una base de datos *XML Nativa* asume el **documento como la**

unidad mínima de almacenamiento y lo modela internamente mediante una **estructura jerárquica de árbol de nodos**.

La gran ventaja técnica de los motores nativos es que **preservan el formato original del documento** sin alteraciones, confiriendo una agilidad excepcional en consultas de datos jerárquicos complejos. Como contrapartida o desventaja, este tipo de almacenamiento puede resultar notablemente **menos eficiente cuando se trabaja con volúmenes masivos de datos puramente planos o relacionales**. Para mitigar esta ineficiencia y **acelerar de forma directa las consultas estructurales** se recurre a la creación y mantenimiento de **índices XML**.

A nivel organizativo, los documentos XML se agrupan de forma lógica en **colecciones**, las cuales poseen la característica de ser organizadas de manera **jerárquica** (enfoque implementado con éxito por el SGBD **eXist-db**). Cada documento dentro de una colección se identifica de forma inequívoca mediante un **identificador único**. Además de texto estructurado, estos sistemas proveen funciones para almacenar objetos complejos serializados catalogados al **guardar como binarios**.

2.2 LENGUAJES DE MANIPULACIÓN XML: XPATH Y XQUERY

Para la recuperación de información desde nodos XML se emplean dos lenguajes de consulta definidos por el consorcio W3C:

- **XPath**: Lenguaje de navegación jerárquica que actúa como un **subconjunto directo de XQuery**, enfocado en el recorrido de rutas de nodos.
- **XQuery**: Lenguaje de consulta completo estructurado específico para XML. Destaca por incorporar las potentes **expresiones FLWOR** (For, Let, Where, Order by, Return), propiedad que no solo filtra datos sino que permite **construir y dar forma a nuevos documentos XML** resultantes a partir de los datos originales. Para iniciar una consulta dentro de un origen XML nativo se requiere invocar la instrucción nativa **db:open()**, cuya función inequívoca es abrir un documento o base de datos XML en la sesión de ejecución.

2.3 GESTIÓN LOCAL CON BASEX Y CONECTIVIDAD ESTÁNDAR XQJ

BaseX es un gestor de bases de datos XML nativo, open-source y multiplataforma, administrable opcionalmente por la interfaz gráfica **BaseX Interface**. Para interactuar de forma programática con BaseX de manera estrictamente local en Java (sin dependencias de red), la API nativa provee dos componentes esenciales:

- **Context**: La clase que se encarga de instanciar y **gestionar las conexiones y sesiones** con las bases de datos de forma local.
- **Command**: Clase para **ejecutar comandos operativos directos** sobre el almacenamiento local.

Los comandos nativos de administración más relevantes dentro del motor BaseX contemplan:

- **CreateDB / DropDB**: Destinados a crear una nueva base de datos XML y eliminar una base de datos completa de forma irreversible, respectivamente.
- **Close / List**: Comandos enfocados a cerrar la base de datos activa de la sesión y listar la totalidad de las bases de datos disponibles en el sistema.
- **Add / Rename / Optimize**: **Add** inserta un nuevo documento XML en una colección; **Rename** permite renombrar o mover un documento existente; y **Optimize** es un comando crítico cuya función exclusiva es **optimizar el rendimiento general de la base de datos** mediante la reconstrucción y saneamiento de todos sus índices de búsqueda internos. Las excepciones de estos flujos son encapsuladas bajo la clase **BaseXException**.

A nivel de desarrollo de aplicaciones Java corporativas, para evitar depender de APIs propietarias de cada fabricante, se utiliza la especificación estándar **XQJ (Java API for XML Data Binding)**. La característica clave que diferencia radicalmente a XQJ de JDBC es que **XQJ está diseñado específicamente para procesar datos XML nativos** en secuencias jerárquicas en lugar de conjuntos de registros tabulares planos.

El estándar XQJ define las siguientes interfaces de comunicación elementales:

1. **XQDataSource**: Componente responsable de configurar y **gestionar las conexiones** con las bases de datos XML.
2. **XQExpression**: Diseñada para la definición y **ejecución de consultas inmediatas** y estáticas.
3. **XQPreparedExpression**: Interfaz avanzada orientada a la **ejecución de consultas paramétricas**. Gracias a la propiedad intrínseca de **reutilización de plantillas precompiladas**, eleva notablemente el rendimiento en ejecuciones repetitivas y bloquea amenazas de inyección de código.
4. **XQResultSequence**: Componente que representa la **secuencia ordenada de resultados** devuelta tras ejecutar una consulta XQuery, permitiendo la navegación a través de los nodos extraídos.

BLOQUE 3: Programación Orientada a Componentes (JavaBeans) y Gestión de QA

3.1 EL PARADIGMA DE COMPONENTES Y LA ESPECIFICACIÓN JAVABEANS

La **Programación Orientada a Componentes** es un paradigma de ingeniería del software enfocado en la concepción de unidades de software independientes, modulares y reutilizables que pueden ser ensambladas de forma dinámica en diversas aplicaciones con el fin de optimizar costos de desarrollo y simplificar el mantenimiento. Para garantizar la cohesión global del software y la comunicación fluida entre plataformas heterogéneas, estos componentes siguen estrictos **estándares de interoperabilidad** (destacando históricamente el estándar **CORBA**).

como un modelo de componentes diseñado específicamente para asegurar la interoperabilidad completa entre diferentes lenguajes y arquitecturas de red distribuida).

En el entorno de desarrollo Java, esta especificación se materializa bajo el estándar de los **JavaBeans**. Para que una clase convencional sea catalogada como un JavaBean reutilizable y pueda integrarse en entornos de desarrollo, debe acatar de forma estricta tres reglas de diseño:

- Proveer de forma obligatoria un **constructor público libre de argumentos** (constructor vacío) para permitir su instanciación dinámica en tiempo de diseño.
- Garantizar su persistencia e integridad implementando obligatoriamente la interfaz **Serializable**. El mecanismo de **serialización en Java** tiene la función inequívoca de **convertir un objeto en una secuencia de bytes** limpia para hacer posible su almacenamiento físico en disco o su envío a través de canales de red, posibilitando restaurar con exactitud el estado del componente más adelante.
- Exponer sus atributos o **propiedades** empleando métodos de acceso normalizados siguiendo la nomenclatura **get<Propiedad>()** (o **is...()** en tipos booleanos) para la lectura de valores simples, y **set<Propiedad>()** para su modificación.

Para la distribución comercial de los componentes, estos se empaquetan en **archivos JAR** con la ventaja competitiva de **agrupar de forma compacta el código junto a todas sus dependencias necesarias**, agilizando su instalación. Asimismo, la presencia de una rigurosa **documentación interna** en el componente ofrece como ventaja directa **facilitar de manera notable su integración** por parte de desarrolladores externos. El entorno **NetBeans** se posiciona como una herramienta de desarrollo ideal para compilar y empaquetar estos artefactos de software.

3.2 MECANISMOS DE DESCUBRIMIENTO DINÁMICO: INTROSPECCIÓN Y REFLEXIÓN

Para que una herramienta de desarrollo de interfaz gráfica (IDE) pueda interactuar de manera interactiva con un JavaBean recién importado sin acceso previo a su código fuente, el entorno de ejecución Java proporciona soporte basado en dos tecnologías complementarias:

- **Introspección:** Se define de manera exacta como la capacidad del sistema para **obtener información estructural de las propiedades, métodos y eventos de un componente sin necesidad de ejecutarlo**. Esto permite plasmar de manera automatizada su configuración en las paletas visuales del IDE en tiempo de diseño.
- **Reflexión (Reflection):** Es el mecanismo subyacente de bajo nivel que faculta al software para **inspeccionar y modificar dinámicamente la estructura íntima de una clase** (campos privados, firmas de métodos y constructores) en pleno tiempo de ejecución. La propiedad resultante de este enfoque es la ganancia de un alto **dinamismo**, habilitando que las aplicaciones configuren sus módulos y objetos de forma totalmente flexible bajo demanda.

3.3 ARQUITECTURA DE EVENTOS Y CLASIFICACIÓN DE PROPIEDADES

La comunicación e interacción entre diferentes JavaBeans se rige bajo un modelo desacoplado guiado por **Eventos**. Este ecosistema delega las operaciones en dos actores esenciales: el emisor o **Source** (fuente originaria del evento ante una mutación de estado) y el receptor o **Listener** (oyente suscrito que aguarda el evento para desencadenar una acción lógica). La vinculación se formaliza mediante la presencia en el Source de métodos de suscripción explícitos bajo el formato **add<Evento>Listener()**.

Atendiendo a su comportamiento comunicativo, las propiedades de un JavaBean se catalogan en tres tipos:

1. **Propiedad Indexada:** Tipo de propiedad diseñada para gestionar **valores múltiples u ordenados asociados** de manera directa bajo un mismo identificador (arrays o colecciones internas).
2. **Propiedad Ligada (Bound Property):** Aquella propiedad configurada para **informar de forma automática a otros componentes acerca de cualquier cambio en su estado**. Su control técnico se apoya en la clase de utilidad **PropertyChangeSupport** y el método **firePropertyChange()**, que difunde la notificación a los oyentes registrados que implementen la interfaz **PropertyChangeListener**.
3. **Propiedad Restringida (Constrained Property):** Aquella propiedad sensible cuyos **cambios de valor propuestos pueden ser vetados o rechazados por otros componentes** de la aplicación si se determina que la alteración vulnera una regla de negocio. Se implementa mediante el uso del componente **VetoableChangeSupport** y la emisión del evento con el método **fireVetoableChange()**. Cualquier oyente enlazado a través de la interfaz **VetoableChangeListener** tiene la potestad legal de vetar la operación lanzando una excepción controlada, forzando la reversión inmediata de la propiedad a su valor original.

3.4 ASEGURAMIENTO DE LA CALIDAD DEL SOFTWARE (QA) Y ESTRATEGIAS DE PRUEBAS

En la ingeniería de software profesional, la **realización de pruebas rigurosas** se constituye como el pilar fundamental para garantizar la calidad del software, mitigando fallos operativos y asegurando el correcto comportamiento del negocio. El enfoque basado en componentes confiere una ventaja inestimable en este ámbito: **los componentes de software pueden ser probados de manera totalmente individual** y aislada en fases tempranas, garantizando su robustez de forma previa a su integración en el sistema global.

Para certificar la calidad global de una solución de persistencia, el equipo de QA ejecuta una batería de pruebas especializadas:

- **Pruebas de Regresión:** Pruebas críticas destinadas a **asegurar que los nuevos cambios, parches o líneas de código introducidas en el software no afecten negativamente** ni rompan el funcionamiento de otras partes del sistema que previamente operaban de forma correcta.
- **Pruebas de Volumen:** Evaluaciones diseñadas con el fin de determinar con exactitud cuál es la **carga máxima de registros y datos** que el sistema es capaz de procesar y alojar de forma simultánea manteniendo su estabilidad nominal.
- **Pruebas de Estrés:** Estrategia consistente en someter al sistema a **condiciones extremas de sobrecarga o concurrencia límite**, muy por encima de los umbrales normales de diseño, para evaluar el comportamiento del software ante fallos inminentes y analizar su punto de ruptura.
- **Pruebas de Recuperación (Recovery Testing):** Evaluaciones destinadas de forma exclusiva a **validar la integridad de los datos y la capacidad de restauración automática del sistema** para regresar a un estado operativo seguro y normal tras sufrir una caída imprevista de tensión, fallo de hardware o desconexión crítica.
- **Pruebas de Aceptación:** Pruebas finales supervisadas directamente en el **entorno real de producción** del usuario, cuya meta es certificar de forma explícita que el software construido satisface la totalidad de los requisitos especificados por el cliente.